Build: Counter with useReducer

1 min read Updated 23 Apr 2026 Topic 49 of 62

Bookmark Mark done

Problem: Build a counter component with increment, decrement, reset, and set-to-value functionality using useReducer.

What This Tests: useReducer pattern, action types, controlled inputs, dispatch.

jsx

JSX Copy

```jsx
function counterReducer(state, action) {
  switch (action.type) {
    case 'INCREMENT': return { count: state.count + 1 };
    case 'DECREMENT': return { count: Math.max(0, state.count - 1) };
    case 'RESET': return { count: 0 };
    case 'SET': return { count: Number(action.payload) || 0 };
    default: return state;
  }
}

function Counter() {
  const [state, dispatch] = useReducer(counterReducer, { count: 0 });
  const [inputValue, setInputValue] = useState('');

  const handleSet = () => {
    dispatch({ type: 'SET', payload: inputValue });
    setInputValue('');
  };

  return (
    <div>
      <h2>Count: {state.count}</h2>
      <button onClick={() => dispatch({ type: 'DECREMENT' })}>-</button>
      <button onClick={() => dispatch({ type: 'RESET' })}>Reset</button>
```

```
          type="number"
          value={inputValue}
          onChange={(e) => setInputValue(e.target.value)}
          placeholder="Set value"
        />
        <button onClick={handleSet}>Set</button>
      </div>
    </div>
  );
}
```

Key Concepts: useReducer for structured state, preventing negative values with `Math.max`, controlled input for set-value, separating UI state (inputValue) from domain state (count).

Extension: Add step size (increment by 5, 10), add undo/redo with action history.

QUICK CHECK — no marks, just for you.

1. In the Counter lesson, the count is currently 0 and the user clicks the "-" button, dispatching { type: 'DECREMENT' }. What is the resulting count?

- ◯ 0, because the DECREMENT case clamps the result with Math.max(0, ...)
- ◯ 0, because dispatching on a zero count is ignored and the reducer is never called
- ◯ The count stays 0 but React logs a warning about invalid state
- ◯ -1, because the reducer subtracts 1 from state.count unconditionally

Check answer

2. Why does the Counter component keep inputValue in useState instead of storing it inside the useReducer state alongside count?

It separates transient UI state (the text being typed) from domain state (the count) that the reducer owns

- Putting a string in reducer state would make dispatch asynchronous and lose keystrokes
- useReducer state is read-only, so a controlled input's value cannot live inside it
- Controlled inputs are required by React to be backed by useState, never by useReducer

Check answer

3. The input is empty and the user clicks the "Set" button, so handleSet dispatches { type: 'SET', payload: '' }. According to counterReducer, what does count become?

- 0, because Number('') is 0 and the || 0 fallback also resolves to 0
- NaN, because the empty string cannot be converted to a number
- undefined, because action.payload is empty and no count is assigned
- It keeps its previous value, because the SET case rejects an empty payload

Check answer

‹ Previous
Machine Coding Round: Strategy & Setup

Next ›
Build: Dynamic Checkbox with Select All

LEARN MORE

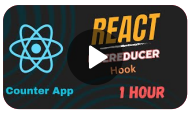
ARTICLES & DOCS

VIDEOS



Counter with useReducer

YouTube



useReducer counter example

YouTube



useReducer vs useState (counter)

YouTube

Hand-picked references for this lesson.